

```

typedef struct Masina {
    int id;
    int nrUsi;
    float pret;
    char* model;
    char* numeSofa;
    unsigned char serie;
} Masina;

typedef struct Heap {
    int lungime; // lungimea totala
    Masina* vector; // pointer la un vector de Masina
    int nrMasini; // cate masini vizibile sunt
} Heap;

Masina CitireMasinaFisier (const char* numef)
{ ... }

void AfisareMasina (Masina m)
{ ... }

```

① Funcția de inițializare a unui Heap:

```

Heap initHeap (int lungime) {
    Heap heap;
    heap.lungime = lungime;
    heap.nrMasini = 0;
    heap.vector = (Masina*) malloc (sizeof(Masina)*lungime);
    return heap;
}

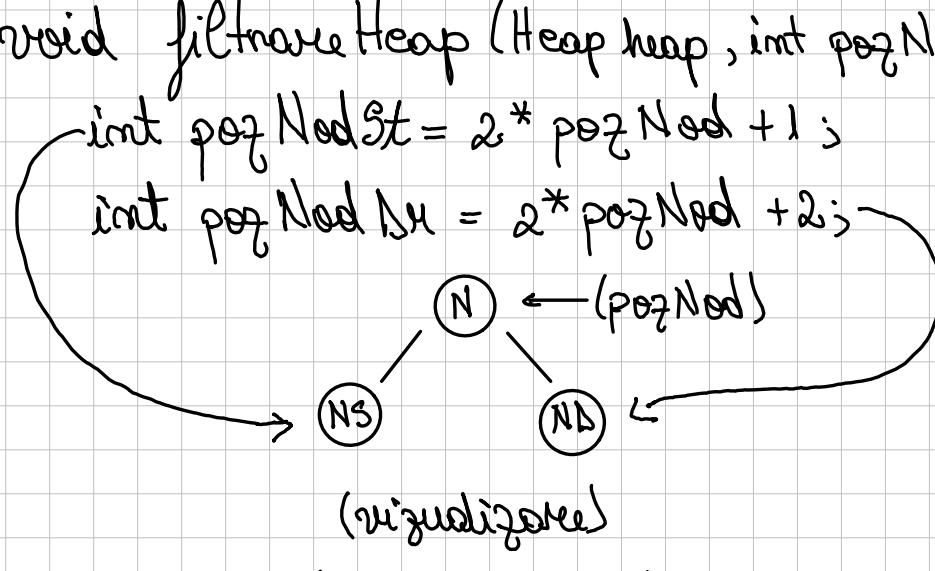
```

② Funcția de filtrare a unui Heap (sau a unu de forat):

```

void filtrareHeap (Heap heap, int pozNod) {
    int pozNodSt = 2*pozNod + 1;
    int pozNodDr = 2*pozNod + 2;

```



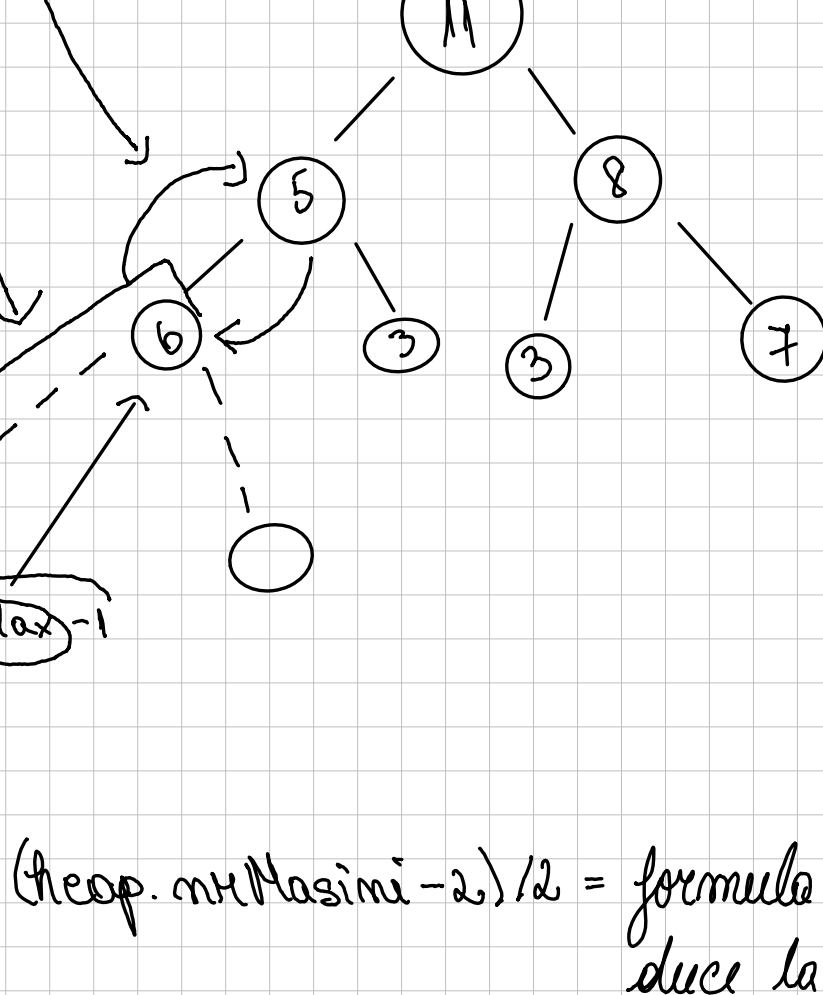
```

    int pozMax = pozNod;
    • ne dorim să prioritizăm poziția maximă.
    if (pozNodSt < heap.nrMasini &&
    • verificăm să nu intrăm peste elementele ascunse.
    heap.vector[pozNodSt] > heap.vector[pozMax])
    { pozMax = pozNodSt; }

    if (pozNodDr < heap.nrMasini &&
    heap.vector[pozNodDr] > heap.vector[pozMax])
    { pozMax = pozNodDr; }

    if (pozMax != pozNod) (avem modificări)
    {
        Masina aux = heap.vector[pozMax];
        heap.vector[pozMax] = heap.vector[pozNod];
        heap.vector[pozNod] = aux;
        if (pozMax <= (heap.nrMasini-2)/2)
            filtrareHeap(heap, pozMax);
    }

```



$(\text{heap.nrMasini}-2)/2 =$ formula de a ne duce la ultimul părinte

$2 * \text{pozMax} + 1 \leq \text{heap.nrMasini} - 1$
 ↳ toate masinile vizibile
 (1) deoarece începe de la 0

$\text{pozMax} \leq (\text{heap.nrMasini}-2)/2$
 aceasta este condiția pentru ultimul părinte

! Prin recursivitate vom ajunge să parcurgem tot arborele.

```

Heap citireHeapDeMasiniFisier (const char* mf) {
    FILE* fptr = fopen(mf, "r");
    Heap heap = initHeap(10);
    while (!feof(fptr)) {
        heap.vector[heap.nrMasini++] = citireMasinaFisier(fptr);
    }
    fclose(fptr);
    • avem heap-ul încercat cu masini.
    for (int i = (heap.nrMasini-2)/2; i >= 0; i--) {
        filtrareHeap(heap, i);
    }
    return heap;
}

```

```

void afisareHeap (Heap heap) {
    for (int i = 0; i < heap.nrMasini; i++) {
        AfisareMasina(heap.vector[i]);
    }
}

```

```

void afisareHeapAscuns (Heap heap) {
    for (int i = heap.nrMasini; i < heap.lungime; i++) {
        AfisareMasina(heap.vector[i]);
    }
}

```

```

void dezalocareHeap (Heap* heap) {
    for (int i = 0; i < heap->lungime; i++) {
        free(heap->vector[i] -> model);
        free(heap->vector[i] -> numeSofa);
    }
    free(heap->vector);
    heap->lungime = 0;
    heap->nrMasini = 0;
    heap->vector = NULL;
}

```

```

Masina extragereMasiniDinHeap (Heap* heap) {
    Masina aux = {0, 0, 0, NULL, NULL, 0};
    if (heap->nrMasini > 0) {
        aux = heap->vector[0];
        heap->vector[0] = heap->vector[heap->nrMasini-1];
        heap->vector[heap->nrMasini-1] = aux;
        heap->nrMasini--;
        for (int i = (heap->nrMasini-2)/2; i >= 0; i--) {
            filtrareHeap(heap, i);
        }
        // filtrăm tot heapul iarăși, după extragere
    }
    return aux;
}

```

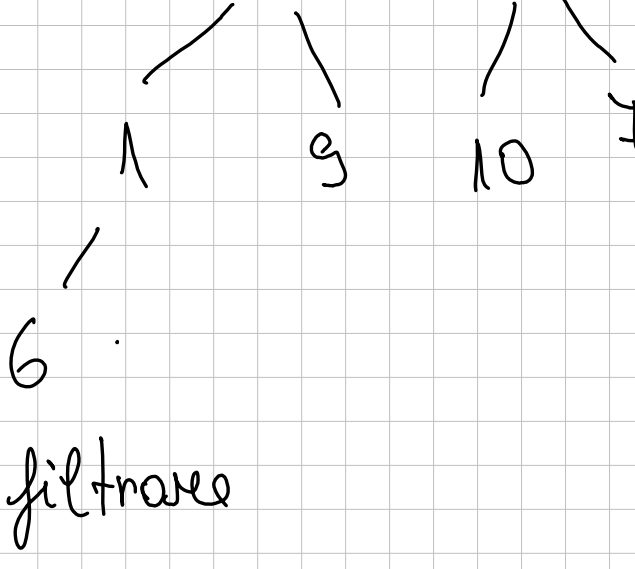
Exercițiu:

lungime = 10
 nrMasini = 8
 vector = 10 masini

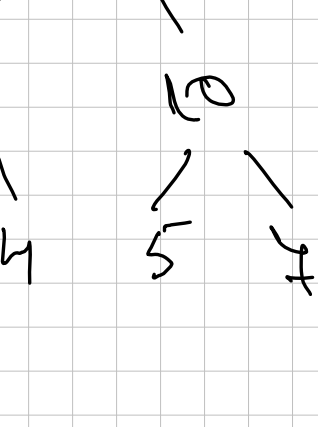
 care e ultimul (x) = ?

12	4	5	1	9	10	7	6	3	1
----	---	---	---	---	----	---	---	---	---

vizibile



filtrare



12, 9, 10, 6, 4, 5, 7, 1
 0 1 2 3 4 5 6 7
 ultimul părinte = $(8-2)/2 = 3$